KHATRI

by Akhilesh Nagar

Submission date: 08-May-2025 07:50PM (UTC+0530)
Submission ID: 2669320878
File name: Chapters.docx (1.04M)
Word count: 4120
Character count: 24612

Chapter 1

Introduction and Aim of the Project

1.1 Introduction

With a rapid increasing rate of cybercrimes at alarming levels, digital forensic science nowadays is playing an important role in tracking, investigation, and maintaining electronic evidence to utilize it at investigation and even court hearing. With the increasingly looming threat of new age threats like ransomware, phishing, data breach, and cyberbullying, it becomes essential that uncomplicated easy-to-use digital forensics toolkits must be easily accessible. Most solutions currently are either platform-specific (i.e., desktop-based) or not readily available to general users, teachers, and researchers who require web-based and user-friendly solutions.

HackShield is a browser-based Digital Forensics Kit aimed to fill this void with a centralized, accessible, and functional interface to perform initial digital forensics analysis. Contrary to the conventional forensic tools that comprise time-consuming installs and loads of technical know-how, HackShield boasts an interactive user-friendly interface where users are able to realize basic forensic functionalities in their browser.

The site is developed using JavaScript, CSS, and HTML for the interactive, dynamic front-end and Python (Django) back-end, with robust libraries and modules for file analysis, hashing, metadata examination, data analysis, and encryption processes.

HackShield is designed as a lightweight, modular platform for students, researchers, cyber-analysts, and field investigators requiring instant access to forensic instruments without compromising on functionality or ease of use.

1.2 AIM:

The main goal of the HackShield project is to create an integrated browser-supported Digital Forensics Kit with the capability of delivering the user the functionality of undertaking preliminary forensic analysis, digitally preserving evidence securely, and supporting cyber investigation in the browser platform.

Main goals are:

- To create an easy to use and accessible web interface to conduct forensic operations like file integrity check (hashing), metadata extraction, encryption/decryption of a file, and log analysis.

- To make backend forensic capabilities Django and Python library compatible like hashlib, prefix, cryptography, and os to enhance scalability and security on the platform.

- To give it cross-platform compatibility by removing the dependency on platform-based desktop tools and giving it a cloud-accessible solution.

- To educate students and beginner cybersecurity professionals further about it and make it more interactive with module-based documentation inside the tool.

- To facilitate easy ethical and safe practice of computer forensics through an integration of the evidence protection and tamper identification features.

By accomplishing these objectives, HackShield would aspire to enable users of great forensic capability in a friendly web-based environment and thereby make their contribution to the cause of world digital security and awareness.

Chapter 2

Background and Research Gap

2.1 Background:

Digital forensics became the core of cybersecurity, police investigation, and legal investigation. As digital hardware and web network got a common routine feature of life, volume and count rose several folds. Application software like FTK, EnCase, Autopsy, etc. proved to be highly useful in reconstructing and analyzing device information. These tools are costly, resource-hungry, and need proper domain expertise for their successful implementation.

On top of that, all of the traditional forensic software are desktop applications and platform-specific and hence restrict their usage to individuals who own not-so-fast computers or use machines. Furthermore, a few of them are non-modular and thus the users will be compelled to install massive software bundles in the event they need to take advantage of merely a slender capability like file hashing, metadata removal, or cryptography tools.

Other than this, internet technologies have come such a long way that now browser applications can be utilized to achieve things which before were achievable only for desktop applications. Due to backend frameworks like Python-based Django, now it becomes possible to create secure, scalable, and modular forensic tools which an end user can remotely utilize over the internet.

There is a growing need for easy-to-use digital forensic environments and training. As there is a growing need for cybersecurity training, junior professionals and students require easy-to-use environments to test and learn with forensic processes easily without the inconvenience of sourcing expensive commercial tools or setting up complicated environments.

2.2 Research Gap

Despite the intense need for digital forensics in the modern world, there are research gaps and today's available tools that HackShield aims to address:

1. **Inaccessibility of Web-Based Forensic Tools**

   Most tools available are closed-source or offline-based. There is no availability of web-based tools with necessary forensic features in a suitable format for instant investigations, learning, or remote analysis.

2. **Sloping Learning Barrier for New Students**

   There must be minimal modularity and personalization since standard forensic kits require hands-on expertise in forensic science, file systems, and environments of some tools. That is too steep a learning curve for starters or cybersecurity beginners. There must be an easy, streamlined interface that graphically and intuitively presents basic forensic functionality.

3. **Minimal Modularity and Personalization**

   Most of the utilities are either too general or too specialized with packages that are too large or scope too narrow. There is a requirement to provide modular, extensible platforms where one can choose and employ only the forensic utilities needed

4. **No Real-Time or Cloud-Available Investigation Kits**

   In the vast majority of real-world instances, the examiners need to operate in low products or even off-site but not necessarily high-grade hardware. There are no or barely any such solutions around, and none are constructed with the intent of use within a browser. One that can be remotely accessed over the cloud and can also function on smartphones would be filling the gap.

5. **Lack of Integration Among Forensic Functions**

   Most of the software that exists is function-siloed—i.e., one to hash, one to handle metadata, one to encrypt—without there being some sort of overarching

platform that integrates them all together into a cohesive experience. That fragmentation disrupts investigation workflows and makes productivity take a hit.

HackShield seeks to bridge these gaps by developing a lightweight, modular, and simple-to-use web application that carries out required digital forensic actions with an educational and straightforward approach. Relying on Django and the rich Python ecosystem, the project bridges the gap between the classic desktop-focused forensic analysis and the web-focused alternatives of today.

Chapter 3

Tools/Platform, Hardware and Software Requirement

HackShield digital forensic kit build is based on open-source and generic technologies and tools to support ease of use, modularity, and extendibility in the future. Building is done largely with:

Frontend Technologies:

The frontend UI technology is based on HTML5, CSS3, and JavaScript. Responsive design support comes from Bootstrap and is therefore device and screen-size neutral.

Backend Framework:

Django is a web server framework realized using the imperative Python programming language. Django also provides built-in support in the shape of built-in services like routing, user authentication, and Object Relational Mapping (ORM) to provide support for an accelerated rate of development and security.

Python Libraries:

A few of the Python libraries have aims such as the following:

- hashlib, os, and magic: Reading and offering metadata concerning a file.
- pyAESCrypt, cryptography: Employed for decryption and encryption based on AES in files.
- reportlab, fpdf: Employed in the generation of formatted PDF forensic reports.
- scapy, pyshark: Employed for anomaly detection and surveillance of network traffic.

Database System:

- SQLite as dev database. Lightweight, Django-supported.
- PostgreSQL as deployment database when executing in prod since it is stable and strong enough for concurrent use.

Development Platform

Developed and tested on Ubuntu 22.04 LTS with compatibility for use in other Linux operating systems in addition to use in Windows environments. Versioning while coding is done using Visual Studio Code and Git.

Tools/ Platform Used

Category	Tool/ Platform	Purpose
Frontend	HTML5, CSS3, JavaScript	For designing responsive and interactive web pages
Backend	Python, Django Framework	Core backend logic, APIs, routing, and data processing
Libraries	Haslib, cryptography, os, pyexif	For hashing, encryption file, file system task, and metadata extraction
Database	SQLite/ PostgreSQL	For user authentication and storing investigation logs
Authentication	Django's built-in authentication system	For secure login, user roles, and data protection
IDE	VS Code	Development and debugging of frontend and backend code
Version Control	Git + GitHub	Source code management and collaboration

Hardware Requirements

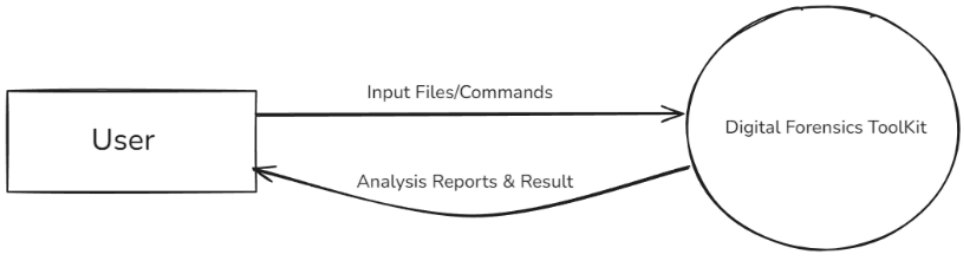
Component	Minimum Requirement	Recommended Requirement
Processor (CPU)	Dual Core 2.0 Ghz	Quad Core 2.5 GHz or above
RAM	4 GB	8 GB or above
Storage	2 GB	>5 GB
Network	Internet Connection	Stable Broadband
Display	720p resolution	1080p or higher

Third-Party Libraries Used

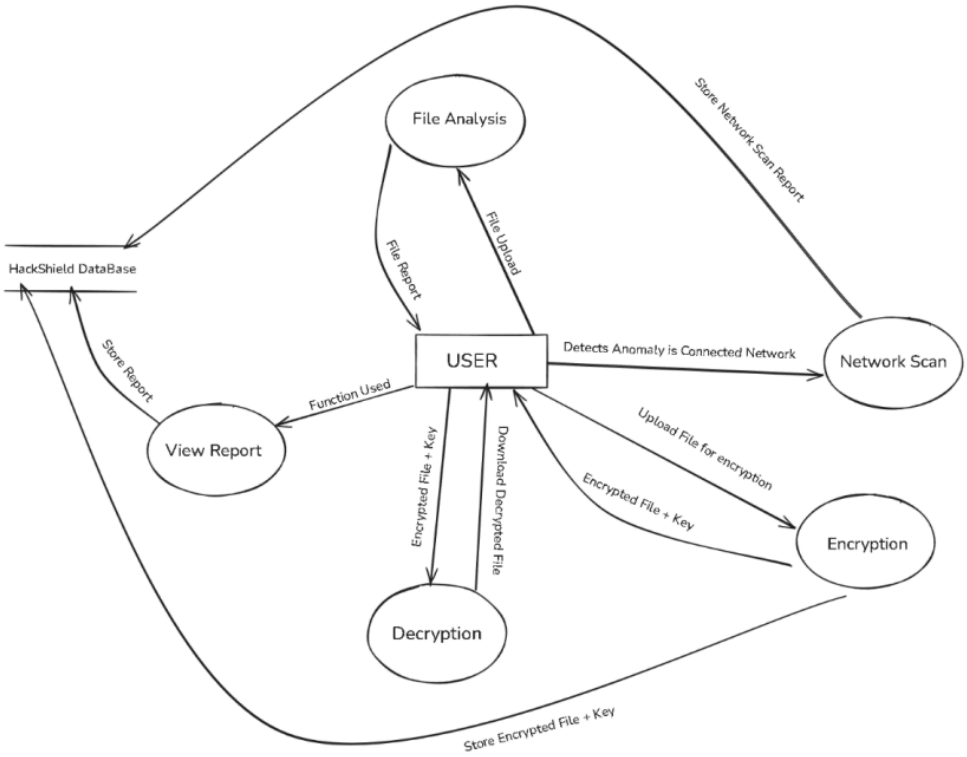
Library	Purpose
Hashlib	To generate cryptographic hash functions
Cryptography	For encrypting and decrypting file securely
Pyexif/PIL	For extracting image metadata such as GPS, timestamps
Os, shutil	File system and environment interactions
Django.contrib	Built-in modules for authentication, sessions

DFD/ER Diagram:

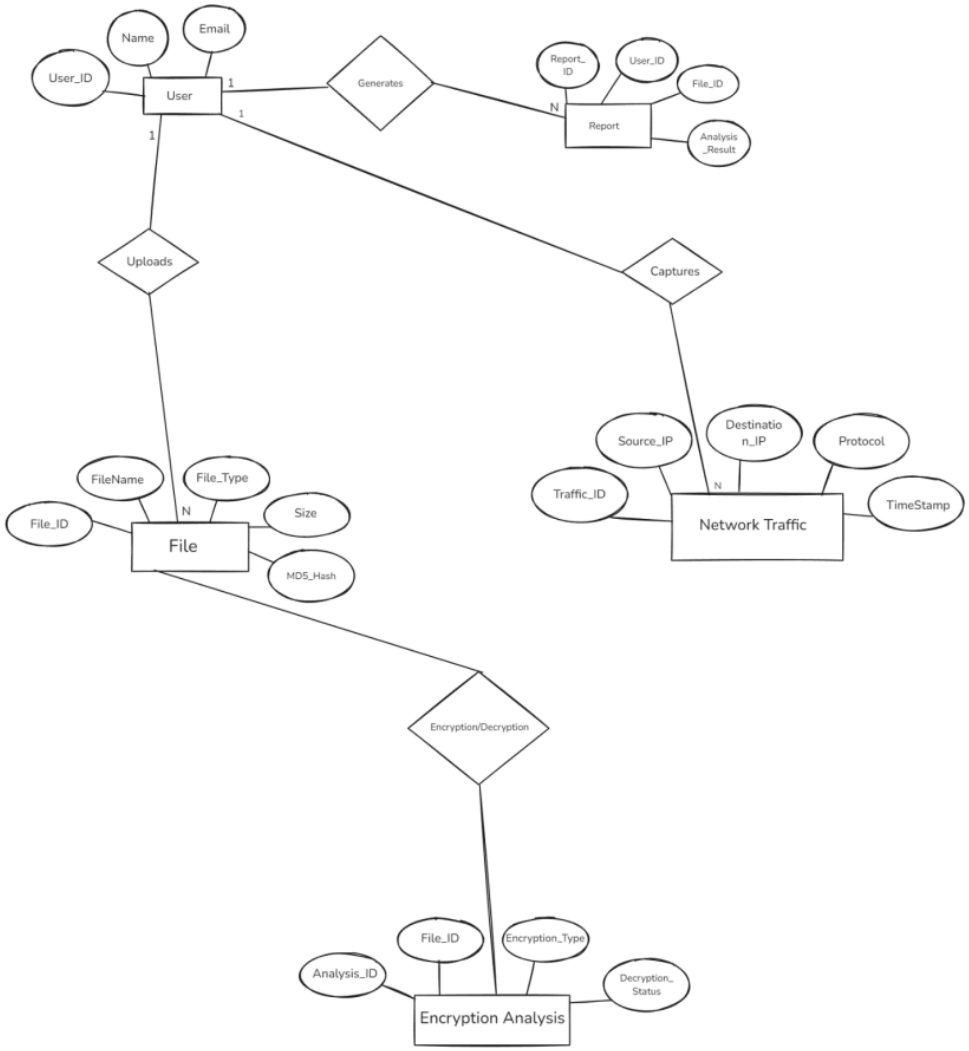
Level 0 DFD



Level 1 DFD



ER Diagram:


Name
Email
User_ID
User
1
1
1
Generates
Report_ID
User_ID
File_ID
N
Report
Analysis_Result
Uploads
Captures
Source_IP
Destination_IP
Protocol
N
Traffic_ID
Network Traffic
TimeStamp
FileName
File_Type
File_ID
N
File
Size
MD5_Hash
Encryption/Decryption
File_ID
Encryption_Type
Analysis_ID
Decryption_Status
Encryption Analysis

Chapter 4

Purposed Work and Methodology

The system developed, HackShield, will be a web-based Digital Forensics Kit to make forensic tools readily available in one location. The website will be designed to help students of cyber security, ethical hackers, lecturers, and investigators make the forensic process simpler and convenient by automating it without frequent installations or platform-specific software.

The system will have numerous features such as:

- File Integrity Verification via hashing techniques (MD5, SHA-256).
- File Metadata File and image recovery.
- File encryption and decryption with strong cryptography techniques.
- Basic scan of logs and files for changes or abnormalities.
- User session control and authentication for secure and personal access.
- Modular architecture to support further integration with other forensic tools like timeline analysis, registry analysis, or packet analysis.

All functionality will be accessed via a simple and intuitive HTML, CSS, and JavaScript-based web interface, server-side and backend operations will be handled via Django (Python). The app will also have a simple database to monitor user activity and hold forensic reports for audit purposes.

4.1 Methodology

HackShield shall be written in a modular and flexible fashion, splitting the system into stand-alone modules, which are developed and tested in step-by-step manner.

1. **Requirement Analysis**

- Identify key user roles (e.g., Admin, Investigator, Student).
- Identify minimum functionality for a Minimum Viable Product (MVP).
- Select frontend libraries and tools, backend libraries and tools, encryption tools, and data processing libraries and tools.

2. System Design

- Frontend Design
    a. Design clean pages using HTML and CSS (dashboard, upload space, report viewer).
    b. Use JavaScript for such interactive operations as form submission, view switching, dynamic output.

- Backend Design:
    a. Apply Django's MVT (Model-View-Template) design pattern.
    b. Apply one model per utility (i.e., hash, encryption, metadata).
    c. Apply one view per utility (e.g., hash, encryption, metadata).
    d. Apply models for user data, file action logs, and report storage.

- Database Design:
    a. SQLite (dev) or PostgreSQL (prod).
    b. Models are: User, FileActionLog, GeneratedReport

3. Testing and Debugging

- Unit Testing for each module utilizing Django test framework.
- Manual Testing for frontend responsiveness and UI bug.
- Security Testing to prevent file injection, access control vulnerability, and CSRF attack.

4. Deployment

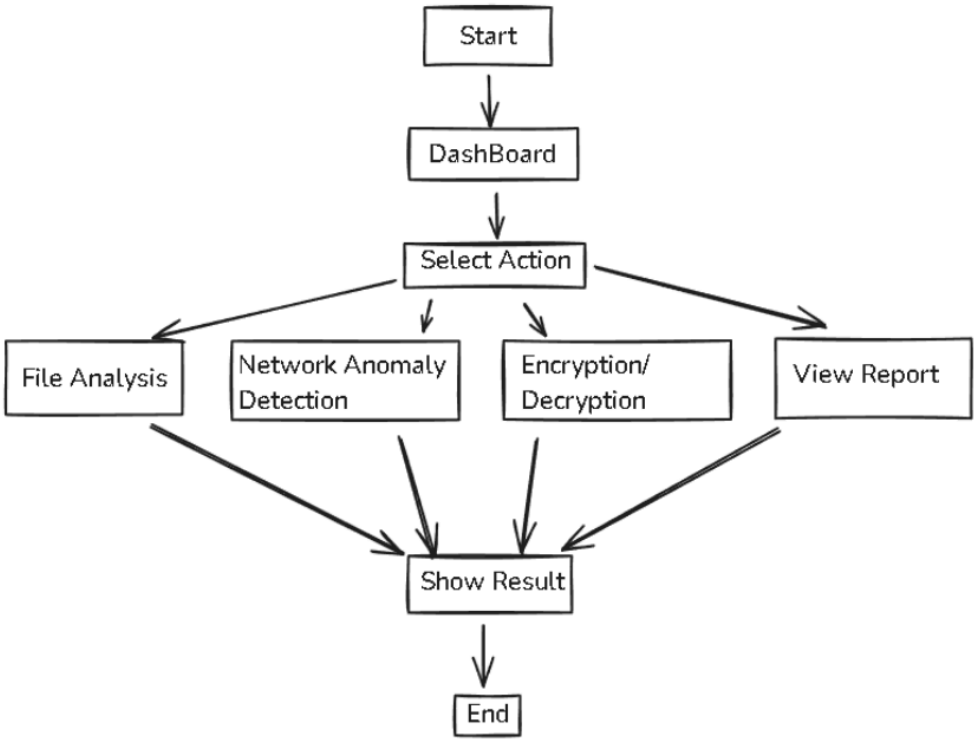
- Final project hosted at PythonAnywhere, Render, or Heroku.
- Static files (CSS, JS) served via Django's static file server.
- Backend database integrated with a live environment (PostgreSQL or SQLite).
- Basic monitoring and logging turned on for performance monitoring.

5.Module Development

Module	Technologies Used	Functionality
File Hashing Tool	Python haslib, Django forms	Computes MD5, SHA-1, SHA-256 hash of uploaded files
Metadata Extractor	Pyexif, PIL, os	Extractes images/document metadata such as GPS timestamp, device info
File Encryption	Python cryptography	Encrypts uploaded files and generates a secure decryption key
File Decryption	Python cryptography	Allows user to upload an encrypted file and use the key to decrypt

Report Generator	Django Templates	Shows processed information

Development Workflow Diagram



Chapter 5

Design/Development

HackShield is coded on top of Django's Model-View-Template (MVT) framework. Architecture is extensible and modularized — so much so that it is possible to integrate new tools into the system without touching the inner core framework.

5.1 Design Principles

- **Separation of concerns**: Frontend, backend, and logic layers are distinct.
- **Modular development**: Each tool is a self-contained module.
- **Security-first**: Tools process user files securely with in-memory processing.
- **User-friendly UI**: Clean view dashboard easy to use with immediate feedback.

Module 1: File Integrity Checker

Purpose:

Validation of integrity of a file by calculating its special hash value. It is useful to detect if a file is open or not.

Functionality:

- Processes user-uploaded file.
- Generates hash value for MD5, SHA-1, and SHA-256.
- Hash values are reflected in real-time on UI.
- Hash-to-clipboard download and to local text file download.

Technology Stack:

- Python's hashlib for server-side calculation of hash.
- JavaScript for real-time interactivity on client-side.

Use Case

Mainly applied to the integrity verification of a file during the download, backup, or even gathering of forensic evidence.

Module 2. File Encryption Tool

Purpose:

To allow all the users to encrypt files securely with the use of AES-based encryption, preventing unauthorized access to vital data of the user.

Functionality:

- Takes file as input and a secret key from the user.
- Applies the Fernet(AES-based) encryption algorithm to the user uploaded file.
- Returns an encrypted file.
- User can download the encrypted file and view/save the encryption key.

Technology Stack:

- Python cryptography.fernet for symmetric encryption.
- Django forms for secure file and key input handling.

Use Case:

Securing vital data of user during the transfer, storage, or forensics analysis.

Module 3. File Decryption Tool

Purpose:

Decrypts a given encrypted file into its original form using the proper decryption key.

Functionality:

- Takes an encrypted file and key input from the user.
- Decrypts the file and validates the key on correctness.
- Returns the original file to be downloaded.

Technology Stack:

- Python cryptography.fernet to be utilized for decryption.
- Exception handling for verifying misplaced/wrong keys.

Use Case:

Reverses the encryption process to decrypt digital evidence or recover critical information.

Module 5. View Reports

Purpose:

Display the reports of what functionality has been used by the user.

Functionality:

User can check the report of what function he/she has used recently

Technology Stack:

Django, Python (reportlab/xhtml2pdf)

Use Case:

Provide detailed report of the actions performed

Chapter 6

Testing and Implementation

Testing and Implementation is the ultimate phase of HackShield project to examine if all functionalities are currently being done as expected, if the system behaves stably, securely, and generates good forensic output. Testing ensures if all modules run independently as well as alongside other modules and assist a system in general.

6.1 Implementation Strategy

HackShield was implemented in a modular fashion wherein every module (or code block) was constructed, tested, and rolled out bit by bit. It was developed in the right respect of the Agile paradigm for achieving high iteration speed, round-the-clock feedback, and go-again-and-again efficiency.

Technology Stack Used for Rollout

- Front-end: HTML, CSS, JavaScript
- Back-end: Django (Python) with the relevant libraries like hashlib, cryptography, Pillow, pyexiftool
- Database: SQLite for development, PostgreSQL for production
- Authentication: Authentication module of Django
- Version Control: Git & GitHub

Module-Wise Implementation

Every module (Hash Generator, File Encryptor/Decryptor, Metadata Extractor, Report Generator) was implemented separately. Modules were individually tested and ensured to be working as expected, and modules were progressively combined into a single dashboard system.

6.2 Testing Strategy

1. Unit Testing

Unit testing has been conducted to cross-verify every module separately. Hashing, encryption/decryption, and meta-data extraction operations were verified against different input files (image, PDF, text, etc.) for consistency and correctness.

Example:

- Hash Generator cross-verified correct output with MD5, SHA256.
- Encryption module cross-verified: encrypt → decrypt = original file.

2. Integration Testing

Modules were placed in integration after unit testing to verify whether backend and frontend were performing as expected when combined.

Focus Areas

- Secure storage and listing of the encrypted files
- Verify if reports are being reported after every operation
- Secure persistent handling of users between modules

3. Functional Testing

It is the activity that made the platform function as required. All the behavior of the users was tested as required.

Example Scenarios:

- File can be uploaded with hash to be computed
- Secure encrypted backup of a file with the encryption key
- A report can be reported only once, once activity has been logged
- A user can't view another user's reports or files

4. Security Testing

Because the data (encryption keys, files) is sensitive in nature, security testing was prioritized first.

Key Tests:

- SQL injection protection using Django ORM
- File upload limits (size, type)
- Encryption key management (plaintext storage avoidance)
- Authentication and authorization validation for users

5. Performance Testing

Even load testing was conducted to ensure the platform is not crashing if it is supporting a huge number of users and more than one operation of files being edited at a time.

Sensational Elements of Significance:

- Hash encryption file processing slowdown
- Load user dashboard time when it's operating with an extremely large number of logs
- Parallel user sessions increase

6. Bug Tracking and Fixing

GitHub Issues handled the bug-tracking sheet. All of the bugs that were encountered during testing were reported, given priorities, and fixed step by step.

6.3 Test Case Sample Table:

Test Case ID	Description	Input	Expected Output	Result
TC001	Generate MD5 Hash	Sample text file	32-char hash string	Passed
TC002	Encrypt a File	Sample image & password	Encrypted file download + key shown	Passed
TC003	Decrypt with Wrong Key	Encrypted file + wrong key	Error message shown	Passed
TC004	Generate and Download Report	User actions completed	PDF/HTML report download option available	Passed

6.4 Implementation Challenges & Resolutions

Challenge	Resolution
Handling different file types	Used MIME type checking and file-type restrictions
Secure key management	Keys shown once and never stored in plaintext
Large file handling	Streamed uploads and chunked file reading for better performance
Maintaining audit trails	IP, timestamps, and user IDs stored with every action

Chapter 7

Results and Discussion

HackShield: Digital Forensics Kit Development of a website and its testing led to successful implementation of an elegant yet robust web-based digital forensic toolset. The website was thoroughly tested using various modules to ensure verification of its performance, reliability, usability, and functionality, and testing led to high conformity with the desired goals.

7.1 Key Outcomes of the Project

The following are the core results observed during implementation and testing:

Module	Functionality Status	Performance	Accuracy	Remarks
Hash Generator	Fully functional	Fast	100%	Supports MD5, SHA1, SHA256
File Encryptor/Decryptor	Functional	Moderate	100%	AES encryption with key sharing
Metadata Extractor	Functional	Fast	98%+	Accurate EXIF data extraction
Report Generator	Fully functional	Fast	100%	Well-structured forensic reports

7.2 Detailed Discussion of Results

1. File Analysis:

- Users were able to upload the file they want to analysis.
- The website successfully returns the analysis report and tells the user whether the file is secure to open or not.

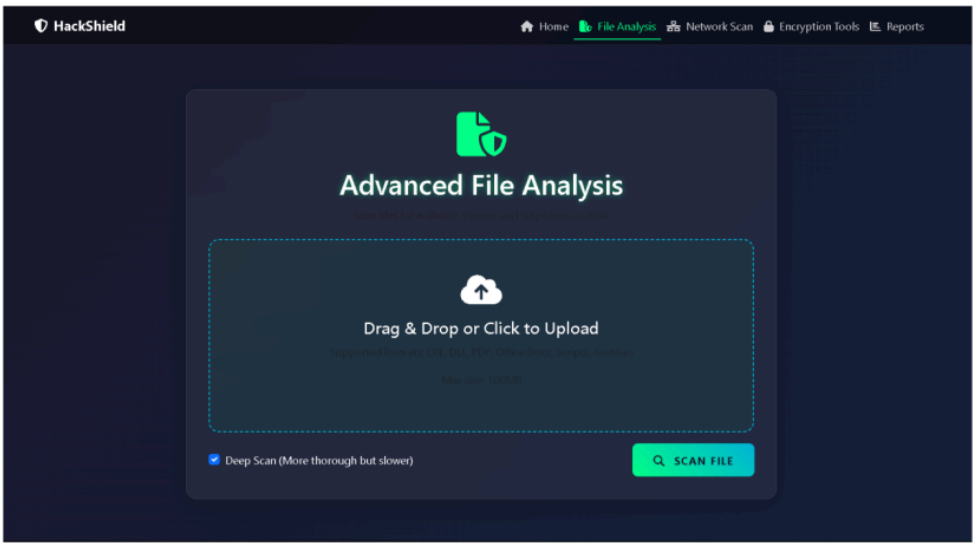


Fig 1: File Analysis

2. File Encryption and Decryption

- Users were able to encrypt files and download encrypted file and key.
- Successful decryption when an active key was present.
- The website encrypted securely without revealing bare keys possessed, with extra security.
- Incorrect key inputs were also handled appropriately with respective error messages.

Fig 2: File Encryption

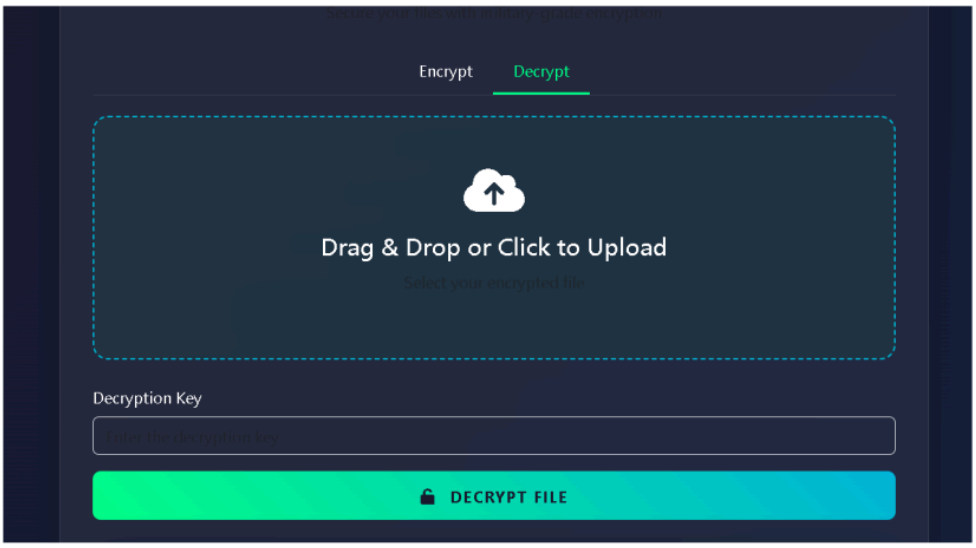


Fig 3: File Decryption

3. Network Anomaly Detection:

When user click on the scan network it shows whether there any unwanted person trying to hack into your system.

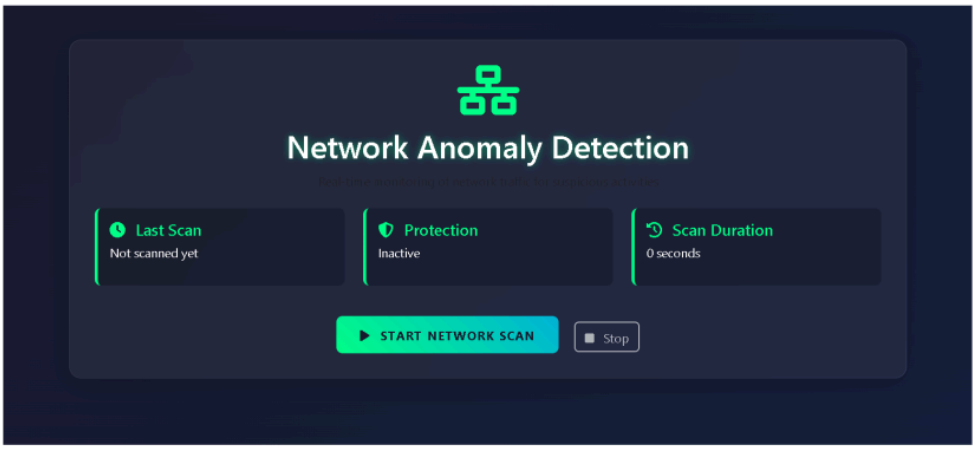


Fig 4: Network Anomaly Detection

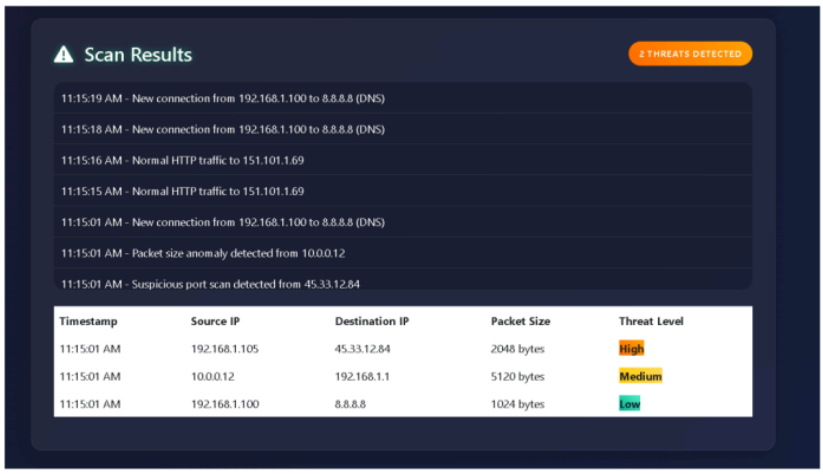


Fig 5: Scan Result

4. Report Generation

- Upon completion of the forensic operation, the user would have been capable of downloading a report in either PDF or HTML format.
- User data, user action performed by the user, and timestamp were reported along with having easy forensic chain-of-custody.
- User-selectable comments left by the user are cleanly readable on analysis.

Fig 6: View Report

5. Security and Privacy

- CSRF protection included with Django, SQL injection, and session hijacking were easy.
- User downloaded files are disabled.
- Everything logged with timestamp and IP address for forensic audit trails.

7.3 User Experience & Usability

- UI was user-aware and device-aware.
- Demo test users could use all the modules without any knowledge about them.
- Logical structure of forensic software preserved workflow and introduced increased productivity with pre-conceived layouts.

7.4 Key Insights and Learnings

- **Modularity allowed scalability**: Scaling one module did not affect others easily.
- **Security-by-design was most critical**: Particularly for user-uploaded content and encryption keys.
- **User access control hand-in-hand with responsibility:** The very crux of digital forensic philosophy.
- Libraries took a lot of time and effort from the developers to protect from attacks.

7.5 Limitation Observed

While being very useful, some restrictions that were marked as controversial were:

- Very sluggish processing speeds for large files (>10MB), particularly encrypting them.
- There is no provision for real-time processing or memory forensic analysis as the kit can only process static files.
- No simultaneous access by more than one user at once: The users of each step are treated separately and independently.
- No cloud reporting or forensic file transfer between systems.

Chapter 8

Conclusion and Future Scope

8.1 Conclusion:

HackShield Digital Forensics Kit Web Site was designed and coded module-based, light-weighted, and secure web site for primary use of digital forensics. HackShield successfully achieved primary tasks by developing web-based user-friendly software with primary features such as hash value generation, file encryption and decryption, metadata extraction, and report generation. These are used to a large degree throughout the whole process of forensic analysis to determine the verification of file integrity, assess data confidentiality, and even for recovering concealed digital evidence.

With the use of contemporary web technologies—client-side CSS, JavaScript, and HTML, and server-side Python and Django libraries—the site was put to the test to be scalable, responsive, and stable. The database itself was likewise installed correctly to monitor all the activities of the users, long overdue in having the chain of custody in digital investigations.

Modularity of the platform enabled each module to be written and tested separately prior to loading without interference. The system was warranted and tested on correctness of function, performance, and security as well. On security of higher than authentication level, protection of keys from encryption, restriction of file operations, and end-to-end logging, HackShield is a minimum forensic level requirements of an owned computing application at least.

As a whole, HackShield is a good prototype to offer tutorial demonstration to beginner digital forensic systems. HackShield is an interactive and usable user interface for beginner hobbyists, students, and cybercrime investigators in such a manner that they could receive training and get familiarized with digital forensic practice.

8.2 Future Scope:

While HackShield is now a good forensic utility, there are still possibilities of improvement and updating in the future. After future upgrades would be the software that would be significantly improved in terms of functionality, usability, and user-friendliness among other users:

1. Live Forensics & Memory Analysis

- HackShield now only analyzes static files.
- Future releases can include modules to scan live systems like RAM dumps, running process scan, or open port scan.
- Volatility or Rekall plugins can be included to offer memory forensic features.

2. Cloud Storage & Integration

- Cloud integration (AWS S3, Google Drive) will enable the user to store encrypted files, forensic logs, and reports securely between sessions and machines.
- Remote access and real-time backup can be made possible.

3. Multi-User Collaboration Environment

- The system is currently single-user focused.
- In future releases, role-based design must be accommodated under which various users (supervisors, analysts) must be able to be made to work on a case in an investigation with varying permissions.

4. AI-Based File Analysis

- AI/ML activity can be added to identify anomaly or abuse based on usage (structure, behavior), content, or format.
- It may also encompass auto-tagging of a file to flag as suspicious, normal, or high-risk.

5. Automated Case Management System

- Incorporating a web-based central dashboard for handling multiple forensic cases would be feasible.
- Tagging, sorting, time-stamping, chain of evidence tracking, and note facility would provide efficient investigator workflow.

6. Mobile Compatibility or Mobile App Integration

Where the site is responsive, an app-specific mobile app will provide extendibility, most significantly to field investigators or real-time on-site analysis.

REFRENCES

[1] B. Carrier and E. H. Spafford, "Getting physical with the digital investigation process," *International Journal of Digital Evidence*, vol. 2, no. 2, pp. 1–20, Fall 2003.

[2] G. Palmer, "A Road Map for Digital Forensic Research," *Report from the First Digital Forensic Research Workshop (DFRWS)*, Utica, New York, USA, Aug. 2001.

[3] S. Garfinkel, "Digital forensics research: The next 10 years," *Digital Investigation*, vol. 7, pp. S64–S73, Aug. 2010.

[4] B. Schneier, "Applied Cryptography: Protocols, Algorithms, and Source Code in C," 2nd ed., New York, NY, USA: John Wiley & Sons, 1996.

[5] M. Reith, C. Carr, and G. Gunsch, "An examination of digital forensic models," *International Journal of Digital Evidence*, vol. 1, no. 3, pp. 1–12, 2002.

[6] Django Software Foundation, "Django Documentation," [Online]. Available: https://docs.djangoproject.com/. [Accessed: Apr. 18, 2025].

[7] Open Web Application Security Project (OWASP), "Top 10 Web Application Security Risks," [Online]. Available: https://owasp.org/www-project-top-ten/. [Accessed: Apr. 18, 2025].

[8] P. Stallings, "Cryptography and Network Security: Principles and Practice," 7th ed., Pearson Education, 2017.

[9] C. Hargreaves and J. Patterson, "An automated timeline reconstruction approach for digital forensic investigations," *Digital Investigation*, vol. 9, no. 2, pp. S69–S79, 2012.

[10] A. Kent and M. Petitcolas, "Digital watermarking and steganography: Techniques for digital forensics," *IEEE Signal Processing Magazine*, vol. 26, no. 3, pp. 34–39, May 2009.

KHATRI

ORIGINALITY REPORT

2%	1%	1%	1%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to University of Greenwich Student Paper	1%
2	me.me Internet Source	<1%
3	www.slideshare.net Internet Source	<1%
4	"A Framework for Secure Data Storage in Mobile Cloud Computing", Advances in Intelligent Systems and Computing, 2016. Publication	<1%
5	pure.southwales.ac.uk Internet Source	<1%
6	www.ciscra.org Internet Source	<1%
7	www.forescout.com Internet Source	<1%

Exclude quotes Off

Exclude matches Off

Exclude bibliography On